

Collaborative Study Web Application

Pilot Deployment Plan

Centralized coordination with local data processing
for non-technical research collaborators

Document version: 1.0

Date: May 28, 2026

1. Executive Summary

This document describes the recommended deployment architecture for piloting the Collaborative Study Web Application with multiple research sites. The goal is to operate one shared central platform while ensuring that raw genotype data never leaves each collaborator's institution. Non-technical collaborators install a simple desktop application once; all heavy quality control (QC), GWAS summary computation, and PCA run locally. Only privacy-preserving aggregates are transmitted to the central server for meta-analysis, chi-square testing, sample relatedness, and collaboration management.

One-sentence summary: One website hosted centrally for everyone, one small desktop app per collaborator installed once — raw data stays local, only summaries come to the central server.

2. Plain-English Overview (for stakeholders)

We keep one website running on our server that everyone logs into. That is where users sign up, start collaborations, invite partners, and view final results. Collaborators continue to use the website in their browser exactly as they do today.

The only new step for collaborators is installing a small desktop application (a signed installer for Mac or Windows, similar to installing Zoom). They open the app once, paste an invite code from email, register their dataset via a file picker, and then leave the app running in the background.

When a study on the website needs their data, the desktop app performs all heavy work on their own computer: QC filtering (missing data, MAF, HWE), PCA, and GWAS summary statistics. It then sends only safe, aggregated results to our server — per-SNP case/control counts and PCA coordinates. Raw genotype rows never leave their machine.

Our central server combines summary results from every site, runs the final statistical analysis (chi-square meta-analysis, pairwise distances), and displays results on the website to authorized collaboration members.

3. Current State vs. Target Architecture

3.1 Current State (development)

Component	Technology	Role
Frontend	React (Vite), Material UI	User interface; calls Flask API
Central Flask	app.py	Users, collaborations, chi-square, pairwise, LLM summary
Orchestrator	Node.js + Kubernetes	Spawns QC worker pods per request
QC Worker	qc_worker.py + Collaborator_Sidecar	Retrieves raw genotypes from MongoDB; runs QC
Databases	MongoDB Atlas (2 clusters)	Central metadata + per-user raw data stores

3.2 Target State (pilot)

Layer	Location	Responsibility
Central website	Cloud / institutional VM	Auth, collaborations, metadata, aggregated results, final stats
Central API	Same host as website	Job queue, enrollment, result ingestion — no raw genotypes
MongoDB Atlas	Managed cloud	Users, datasets (metadata), collaborations, stats, jobs
Desktop Site Agent	Each collaborator machine	QC, GWAS summary, PCA on local CSVs
Local data	Collaborator filesystem	Raw genotype files; never uploaded by default

3.3 Key Architectural Invariants

- Central server never receives raw genotype rows in the default pilot configuration.
- Local Site Agent uses outbound HTTPS only — no inbound firewall ports required.
- No Kubernetes or Docker required on collaborator machines.
- No terminal, command line, or developer tooling required for collaborators.
- Each site's agent is authenticated with a site-specific token; jobs are scoped per user.

4. Data Sharing Levels

The application already produces three levels of QC output. For the pilot, only Levels A and B are transmitted to the central server by default.

Level	Contents	Risk	Pilot default
A — Aggregates	Per-SNP case/control counts (0/1/2); no sample IDs	Low	Send
B — PCA / privacy	PCA coordinates or privacy-transformed data	Medium	Send
C — Cleaned raw	Post-QC genotype matrix (filtered SNPs)	High	Never (opt-in only)

4.1 What Each Level Enables

Central feature	Data required	Level
GWAS chi-square meta-analysis	Per-SNP aggregated counts	A
Sample relatedness	PCA coordinates per sample	B
Population stratification	PCA coordinates per sample	B
Re-running QC with different thresholds	Cleaned matrix	C (local only)

Privacy statement for pilot sites: "Raw genotypes never leave your computer; we only receive aggregated SNP counts and PCA coordinates."

5. End-to-End Collaboration Flow

5.1 Sign-up and dataset registration

Collaborator logs into the central website. They register dataset metadata (phenotype, sample count). The desktop app discovers the local CSV, computes file hash and dimensions, and posts metadata to the central server without uploading rows.

5.2 Collaboration invitation

Study initiator creates a collaboration on the website and invites partners. Existing JWT-based invitation flow is unchanged.

5.3 QC job enqueue

When all parties accept, the central server enqueues a chained_qc job per participant (Missing → MAF → HWE → PCA) instead of calling the Kubernetes orchestrator.

5.4 Local QC execution

Each site's desktop agent polls for its job, runs QC locally, and posts surviving SNP IDs, sample IDs, and PCA coordinates (Level A/B) back to the central server.

5.5 GWAS summary

Central server enqueues gwas_summary jobs. Each agent computes per-SNP case/control counts locally and uploads aggregates.

5.6 Meta-analysis

Initiator triggers chi-square on the central server. Computation uses only aggregated counts (vectorized NumPy implementation; no raw data required).

5.7 Relatedness and stratification

Pairwise distances computed on centrally stored PCA coordinates from each site.

5.8 AI summary (optional)

Initiator can generate a one-page GWAS summary via LLM using only aggregated, anonymized statistics (Site A, Site B labels — no raw data sent to OpenAI).

6. Local Desktop Site Agent

6.1 Technology Choice

Recommended stack: **PySide6 (Qt)** for the user interface plus **PyInstaller** to produce signed macOS (.app / .dmg) and Windows (.exe / .msi) installers. The entire QC pipeline is already Python; this avoids a second toolchain and keeps maintenance simple. Estimated installer size: approximately 40 MB. No Python installation required on the collaborator's machine.

6.2 Collaborator Onboarding (5 steps)

- Download signed installer from email link (Mac or Windows).
- Install: drag to Applications (Mac) or Next-Next-Finish (Windows).
- Open app, paste invite code from email.
- Click "Add dataset," select CSV via file dialog, enter phenotype name.
- Use the central website in browser for collaborations; app handles processing automatically.

6.3 Application Features

- Main window: connection status, registered datasets, activity log.
- System tray / menu bar icon for background operation.
- Long-poll connection to central server for job delivery.
- Local file reads only from user-selected paths.
- Privacy tripwire: refuses to POST payloads containing raw genotype matrices.
- Auto-update check on startup via central /agent/version endpoint.
- Logs written to OS-standard log directory for support troubleshooting.

6.4 Code Reuse Mapping

Existing code	Role in desktop agent
Collaborator_Server/*.py	QC algorithms (MAF, HWE, Missing, PCA, privacy)
gwas_summary.py	GWAS per-SNP count generation
qc_worker.py action handlers	Lifted to shared actions.py module
orchestrator_client.py	Replaced by JobsClient (agent → central HTTP)

6.5 Code Signing Requirements

Before distributing to pilot sites, installers must be code-signed and notarized (macOS) or signed with an OV/EV certificate (Windows). Without this, operating systems display security warnings that block many non-technical users and trigger IT review.

7. Job Polling and Site Isolation

The desktop agent uses a pull model: it periodically calls `GET /api/agent/jobs/next` with its bearer token. The central server authenticates the token, identifies the associated `user_id`, and returns only jobs tagged for that user. Jobs for other collaborators are never exposed to this agent.

7.1 Enrollment Flow

- Administrator generates an enrollment / invite code tied to a specific user account.
- Collaborator pastes code into desktop app on first launch.
- Central server validates code and issues a long-lived bearer token.
- Token stored locally in app config (e.g. `~/Library/Application Support/CollabStudy/`).
- All subsequent API calls include Authorization: Bearer <token>.

7.2 Job Creation

When a collaboration requires processing from N sites, the central server creates N separate jobs in the jobs collection, each tagged with the corresponding `user_id`. This mirrors the existing per-user dataset binding in `app.py`. Analogy: like email — everyone connects to the same server, but each inbox only receives messages addressed to that account.

8. Central Server Changes

8.1 New Jobs API Endpoints

Endpoint	Method	Purpose
/api/agent/enroll	POST	Exchange invite code for bearer token
/api/agent/datasets	GET	List datasets owned by this site
/api/agent/datasets/{id}/metadata	POST	Register metadata + file hash (no rows)
/api/agent/jobs/next	GET	Long-poll next job for authenticated site
/api/agent/jobs/{id}/result	POST	Upload job results (Level A/B)
/api/agent/version	GET	App version check for auto-update

8.2 Flask Refactoring

- Replace orchestrator.submit_* calls with enqueue_job(...) writes to db.jobs collection.
- Remove central-server reads/writes to CollaboratorDB per-user raw data collections.
- Set USE_ORCHESTRATOR=false for pilot deployment.
- Add MAX_CONTENT_LENGTH and gzip support for agent result uploads.
- Frontend: use VITE_API_URL environment variable instead of hardcoded localhost.

8.3 Environment Variables (Central)

Variable	Purpose
MONGO_URI	MongoDB Atlas connection (collaborative study cluster)
SECRET_KEY	JWT signing (rotate before pilot)
AGENT_ENROLLMENT_SIGNING_KEY	Mint and verify site enrollment codes
ALLOWED_ORIGINS	CORS for production frontend URL
USE_ORCHESTRATOR	Set to false for pilot
OPENAI_API_KEY	Optional: GWAS AI summary for initiators

8.4 Central Hosting Options

Option	Components	Estimated cost	Best for
Simple PaaS	Render/Fly/Railway + Atlas + Cloudflare Pages	\$30-60/month	Fast pilot launch
AWS	ECS Fargate + Atlas + S3/CloudFront	Variable	Production scale
Institutional VM	Docker Compose (Flask + Nginx) + Atlas	Existing infra	IRB / data residency

9. Result Upload Strategy (POST Size)

For typical pilot datasets, a single gzipped HTTPS POST is sufficient. The agent automatically selects the appropriate tier based on payload size.

Dataset scale	GWAS counts (gzip)	PCA coords (gzip)	Tier
471 samples × 3K SNPs	~60 KB	~250 KB	1 — single POST
5K samples × 500K SNPs	~3 MB	~1 MB	1 — single POST
100K samples × 5M SNPs	~30 MB	~15 MB	2 — chunked POST
>100 MB total	—	—	3 — presigned S3/R2 URL

- Tier 1 (≤ 5 MB gzip): Single POST with Content-Encoding: gzip.
- Tier 2 (5–100 MB): Chunked POST with finalize flag; merge server-side.
- Tier 3 (>100 MB): Presigned URL to object storage (Cloudflare R2, Backblaze B2, or S3).
- MongoDB 16 MB document limit: large results stored in qc_results collection with batching (existing pattern).

10. Security and Privacy

- Outbound-only network from site agents — simplifies institutional firewall approval.
- Per-site bearer tokens scoped to own datasets and collaborations only.
- Agent egress filter: refuse payloads containing raw data keys or exceeding size thresholds.
- SHA-256 file hash registration for dataset integrity without uploading file contents.
- Optional TLS certificate pinning in agent for central server.
- Audit log: all job creation, completion, and metadata updates in `audit_events` collection.
- LLM summary: only aggregated, anonymized digest sent to OpenAI; no raw genotypes or sample IDs.
- GDPR/HIPAA narrative: central server holds de-identified aggregates; raw data remains at site.

11. Pilot Rollout Timeline

Week	Focus	Deliverables
Week 1	Central refactor	Jobs API, enqueue_job migration, agent auth, production frontend deploy
Week 2	Desktop agent	actions.py extraction, PySide6 app, PyInstaller, code signing pipeline
Week 3	Pilot validation	E2E on eye_color dataset; first external site; audit log review

11.1 Validation Criteria

- Chi-square results match current development environment on same dataset.
- Pairwise distance matrices match within numerical tolerance.
- No raw genotype documents written to central MongoDB during pilot flow.
- Collaborator completes onboarding without terminal or Docker.
- Job isolation verified: Site A agent never receives Site B jobs.

12. Decisions Required Before Implementation

#	Decision	Recommendation
1	Privacy posture: send only Levels A + B?	Yes — never raw rows by default
2	OS coverage for pilot installers	macOS + Windows; add Linux if required
3	Central hosting provider	Choose based on IT constraints
4	Retire K8s orchestrator for pilot?	Yes — simplifies collaborator experience
5	Local dataset file naming convention	e.g. data/<phenotype>.csv

13. What Stays vs. What Changes

Category	Keep unchanged	Change for pilot
Algorithms	MAF, HWE, Missing, PCA, privacy, GWAS summary	summary, chi-square
UI	React collaboration flows, GWAS tables, AI search	API only, config only
Central DB schema	collaborations.stats, surviving_snps, chi_square_results	
Infrastructure	—	Job queue replaces orchestrator; raw data off central Atlas
Collaborator UX	—	Desktop app replaces K8s/Docker/terminal